Learning Objectives

By the end of this session, you should be able to:

- Explain how control structures can be implemented using conditional and unconditional jumps.
- Extend a code generator to support labels, jumps, and conditional branches.
- Extend the grammar and parser to support `if`/`else` statements and `while` loops.
- Extend expression trees and code generation to support comparison operators (`==`, `!=`, `<`, `<=`, `>`, `>=`).

Review — Back to Session 3: Control Flow with Jumps

Remember Session 3? Back then, we explored how control structures such as `if` statements and `while` loops can be expressed using conditional and unconditional jumps. Almost twenty sessions later, we are finally ready to put these ideas to work in our own compiler!

Consider the following program, `xtest_review_ws03_if.abc`:

```
global x: i64;

fn main(): i64
{
    x = 42;

    if (x) {
        x = 0;
    } else {
        x = 1;
    }

    return x;
}
```

In Session 3, we learned how to rewrite such a program using `goto` statements and labels.

Create `xtest_review_ws03_if_goto.abc`:

```
global x: i64;

fn main(): i64
{
    x = 42;

    if (x == 0) { goto elseLabel; }
    x = 0;
    goto doneLabel;

label elseLabel:
    x = 1;

label doneLabel:
    return x;
}
```

Both programs have the same behavior. The second version, however, makes the control flow explicit: instead of an `if`/`else` statement, we use labels and jumps to determine which instructions are executed.

This is exactly the representation we need for assembly code generation.

Our next goal is to generate assembly code corresponding to the second program directly, using the code generator we developed in previous sessions.

For this, we will introduce four new operations:

- `genGetLabel()` to obtain a unique label.
- `genDefLabel()` to define a label at the current position.
- `genJump()` to generate an unconditional jump.
- `genJumpIfFalse()` to generate a conditional jump when a value is zero.

Once these operations are available, generating code for `if`/`else` statements becomes a matter of translating the control-flow structure into the appropriate sequence of jumps and labels.

© Michael C. Lehn | CC BY-SA 4.0

Step 1 — From Goto to Code Generation

We have just reviewed how an `if`/`else` statement can be expressed using labels and jumps. Now let's see how we can generate the corresponding assembly code with our own code generator. Create a new file, `xtest_gen_05.abc`, and enter the following test program:

```
@ "gen.hdr"
@ "symtab.hdr"

fn main()
{
    genInit();

    // global x: i64;
    genAddGlobal("x");

    // fn main() {
    genFuncDefBegin("main");

    // x = 42;
    {
        local rVal: GenReg = genConstant("42");
        local lValAddr: GenReg = genLoadAddr("x");
        genStore(rVal, lValAddr);
        genReleaseReg(rVal);
        genReleaseReg(lValAddr);
    }

    // if (x)
    local elseLabel: GenLabel = genGetLabel();
    local doneLabel: GenLabel = genGetLabel();
    {
        local xVal: GenReg = genLoadVar("x");
        genJumpIfFalse(xVal, elseLabel);
        genReleaseReg(xVal);
    }
    // { x = 0; }
    {
        local rVal: GenReg = genConstant("0");
        local lValAddr: GenReg = genLoadAddr("x");
        genStore(rVal, lValAddr);
        genReleaseReg(rVal);
        genReleaseReg(lValAddr);
        genJump(doneLabel);
    }
    // else { x = 1; }
    genDefLabel(elseLabel);
    {
        local rVal: GenReg = genConstant("1");
        local lValAddr: GenReg = genLoadAddr("x");
        genStore(rVal, lValAddr);
        genReleaseReg(rVal);
        genReleaseReg(lValAddr);
    }
    genDefLabel(doneLabel);

    // return x;
    {
        local xVal: GenReg = genLoadVar("x");
        genReturn(xVal);
        genReleaseReg(xVal);
    }
    genFuncDefEnd();
}
```

Study the program and compare it with `xtest_review_ws03_if_goto.abc`. Notice how the new generator functions correspond to labels and jumps. Why do we need two labels? Why is the unconditional jump necessary? What happens when `genJumpIfFalse()` receives zero? And why can we release `xVal` immediately afterward?

The new generator functions are not yet implemented. We will take care of that in the next step.

© Michael C. Lehn | CC BY-SA 4.0

Step 2 — Extending the Code Generator Interface

Before implementing the new operations, we need to declare them. Extend `gen.hdr` with the following declarations:

```
type GenLabel: unsigned;
extern fn genGetLabel(): GenLabel;
extern fn genDefLabel(labelIdent: GenLabel);
extern fn genJump(labelIdent: GenLabel);
extern fn genJumpIfFalse(condVal: GenReg, labelIdent: GenLabel);
```

Here, `GenLabel` represents a unique identifier for a label. Function `genJumpIfFalse()` generates a conditional jump if `condVal` is zero.

Next, add a stub for each new function to `gen.abc`. For now, each function body should contain only:

```
assert(0 && "TODO");
```

Make sure that the project compiles. The new functions are not supposed to work yet — we will implement them step by step.

Step 3 — Implementing Jumps and Labels: Assertion Chasing

Let's implement the missing generator functions using assertion chasing. Run `xtest_gen_05` and follow the assertions one by one. Each assertion tells us which function to implement next.

Tasks

1. **The first assertion comes from `genGetLabel()`.** We need a way to assign a unique identifier to each label. Add a global variable `labelCount` to `gen.abc` and implement:

   ```
   global labelCount: GenLabel;

   fn genGetLabel(): GenLabel
   {
       return labelCount++;
   }
   ```

   The counter keeps track of how many labels have already been created. Notice the postfix increment: the first label gets `ID 0`, the second `ID 1`, and so on. Yes, we did that on purpose! ;-)

2. **Run the test again.** The next assertion comes from `genJumpIfFalse()`. Remember how conditional jumps work on our ULM architecture? Subtracting zero from a register and writing the result to register `%0` updates the status flags without changing the value we want to test. A subsequent `jz` jumps if the zero flag (`ZF`) is set.

   Implement:

   ```
   fn genJumpIfFalse(condVal: GenReg, labelIdent: GenLabel)
   {
       setSegment(TEXT);
       printf("\tsubq \t0, \t%%%d, \t%%0\n", condVal);
       printf("\tjz .L%d\n", labelIdent);
   }
   ```

   Notice that we use `.L0`, `.L1`, and so on as assembly label names. The numeric part comes directly from `genGetLabel()`.

3. **The next assertion comes from `genJump()`.** This one is straightforward: generate an unconditional jump using `jmp`.

   ```
   fn genJump(labelIdent: GenLabel)
   {
       setSegment(TEXT);
       printf("\tjmp .L%d\n", labelIdent);
   }
   ```

4. **Finally, the assertion from `genDefLabel()` remains.** We simply emit the label definition:

   ```
   fn genDefLabel(labelIdent: GenLabel)
   {
       setSegment(TEXT);
       printf(".L%d:\n", labelIdent);
   }
   ```

5. **Run the test again, assemble and execute the generated code.** Check the exit code. Then change the initial value of `x` from `42` to `0` and repeat. Does the result change as expected?

© Michael C. Lehn | CC BY-SA 4.0

Step 4 — Adding If Statements to Our Compiler

Our code generator can now generate conditional and unconditional jumps. Let's use this functionality to extend our compiler with `if` statements. Our goal is to compile and execute the following program, `examples/ws22_if.not-abc`:

```
global x;

fn main()
{
    x = 42;
    if (x) {
        x = 0;
    } else {
        x = 1;
    }
    return x;
}
```

First, extend our grammar to support if statements with an optional `else` branch:

```
top-level     = { global | fn } EOI
global        = "global" identifier { "," identifier } ";"
local         = "local" identifier { "," identifier } ";"
fn            = "fn" identifier "(" [ identifier { "," identifier } ] ")"
                compound-stmt
compound-stmt = "{" { expr-stmt | if-stmt | return-stmt | local } "}"
if-stmt       = "if" "(" expr ")" compound-stmt
                [ "else" (if-stmt | compound-stmt) ]
expr-stmt     = expr ";"
```

Notice that the else branch is optional and may contain another `if` statement. This allows us to write `else if` chains without introducing a separate grammar rule.

Tasks

1. Add the following function to `parser.abc`:

```
fn parseIfStmt(): bool
{
    if (token.kind != IF) {
        return false;
    }
    getToken();
    local doneLabel: GenLabel = genGetLabel();
    local elseLabel: GenLabel = genGetLabel();
    expected(LPAREN);
    getToken();
    local expr: -> Expr = parseExpr();
    if (!expr) {
        error("expression expected");
    }
    expected(RPAREN);
    getToken();
    local condVal: GenReg = loadExpr(expr);
    genReleaseReg(condVal);
    genJumpIfFalse(condVal, elseLabel);
    releaseExpr(expr);
    if (!parseCompoundStmt()) {
        error("compound for then branch expected");
    }
    genJump(doneLabel);
    genDefLabel(elseLabel);
    if (token.kind == ELSE) {
        getToken();
        if (token.kind == IF) {
            parseIfStmt();
        } else if (!parseCompoundStmt()) {
            error("compound for else branch expected");
        }
    }
    genDefLabel(doneLabel);
    return true;
}
```

© Michael C. Lehn | CC BY-SA 4.0

2. Integrate `parseIfStmt()` into the existing parser. Decide where a forward declaration is needed and which parsing function must be extended.
3. Compile `examples/ws22_if.not-abc`, assemble the generated code, and execute it. Verify the exit code.
4. Modify the example to test an if statement without an `else` branch. Choose the condition and return value so that you can verify whether the statement was executed correctly.

Look What Our Compiler Can Already Do!

With if statements working, our compiler is already capable of running some surprisingly useful programs. For example, have a look at `examples/ws22_printInt.not-abc`:

```
fn printInt(n)
{
    if (n) {
        if (n / 10) {
            printInt(n / 10);
        }
        putchar(n % 10 + '0');
    } else {
        putchar('0');
    }
}

fn main()
{
    printInt(1234); putchar('\n');
    printInt(1); putchar('\n');
    printInt(0); putchar('\n');
}
```

This program prints integers without using `printf()`! It combines recursion, arithmetic, function calls, and nested `if` statements.

Of course, comparison operators such as `==`, `!=`, `<`, and `>=` would be nice to have. Let's put those on our TODO list.

But since we're already tuning our parser, why not add `while` loops first? ;-)

What's Next?

In the next step, our compiler will be able to compile `examples/ws22_readint.not-abc`:

```
fn printInt(n)
{
    if (n) {
        if (n / 10) {
            printInt(n / 10);
        }
        putchar(n % 10 + '0');
    } else {
        putchar('0');
    }
}

fn readInt()
{
    local n, ch;
    n = 0;
    ch = getchar();
    while (ch - '\n') {
        n = n * 10 + ch - '0';
        ch = getchar();
    }
    return n;
}

fn main()
{
    local n;
    n = readInt();
    printInt(n);
    putchar('\n');
}
```

© Michael C. Lehn | CC BY-SA 4.0

Step 5 — Adding While Loops to Our Compiler

We already know how to generate conditional and unconditional jumps. Adding while loops therefore requires only a small extension to our parser. Extend the grammar from Step 4:

```
compound-stmt = "{" { expr-stmt | if-stmt | while-stmt
                    | return-stmt | local } "}"

while-stmt    = "while" "(" expr ")" compound-stmt
```

Add the following function to `parser.abc`:

```
fn parseWhileStmt(): bool
{
    if (token.kind != WHILE) {
        return false;
    }
    getToken();
    local condLabel: GenLabel = genGetLabel();
    local doneLabel: GenLabel = genGetLabel();
    expected(LPAREN);
    getToken();
    local expr: -> Expr = parseExpr();
    if (!expr) {
        error("expression expected");
    }
    expected(RPAREN);
    getToken();
    genDefLabel(condLabel);
    local condVal: GenReg = loadExpr(expr);
    genReleaseReg(condVal);
    genJumpIfFalse(condVal, doneLabel);
    releaseExpr(expr);
    if (!parseCompoundStmt()) {
        error("compound for while body expected");
    }
    genJump(condLabel);
    genDefLabel(doneLabel);
    return true;
}
```

Tasks

1. **Integrate `parseWhileStmt()` into the existing parser.** As in Step 4, determine where a forward declaration is needed and which parsing function must be extended.
2. **Build and test.** Rebuild the compiler and compile `examples/ws22_readint.not-abc`. Assemble and execute the generated program. Does it behave as expected?
3. `Test the loop.` Try different inputs, including `0` and numbers with several digits. Explain why the condition is evaluated again on every iteration.

Step 6 — Generating Code for Comparison Operators

Our compiler now supports if statements and while loops. But something is still missing: comparison operators such as `==`, `!=`, `<`, and `>=`.

Before extending our parser, let's prepare the code generator. Consider an assignment such as:

```
x = a > b;
```

The comparison `a > b` is an expression. It must evaluate to `1` if the condition is `true` and 0 otherwise. Remember how we implemented control flow using labels and jumps? We can apply exactly the same idea here. Conceptually, the generated code corresponds to:

```
if (a > b) { goto trueLabel; }
x = 0;
goto doneLabel;

label trueLabel:
    x = 1;

label doneLabel:
```

© Michael C. Lehn | CC BY-SA 4.0

Extending the Code Generator. First, extend the `GenInstr` enumeration in `gen.hdr` for constants `GEN_EQ`, `GEN_NE`, `GEN_LE`, `GEN_LT`, `GEN_GE`, and `GEN_GT`.

Add the following helper function to `gen.abc`:

```
fn condInstr(instr: GenInstr): ->readonly char
{
    switch (instr) {
        default: assert(0 && "not handled");
        case GEN_NE: return "jnz";
        case GEN_EQ: return "jz";
        case GEN_LE: return "jle";
        case GEN_LT: return "jlt";
        case GEN_GE: return "jge";
        case GEN_GT: return "jgt";
    }
}
```

Now extend the switch statement in `genInstr()` with the following cases:

```
fn genInstr(instr: GenInstr, left: GenReg, right: GenReg): GenReg
{
    // ... as before
   switch (instr) {
        case GEN_NE:
        case GEN_EQ:
        case GEN_LT:
        case GEN_LE:
        case GEN_GT:
        case GEN_GE:
        {
            local doneLabel: GenLabel = genGetLabel();
            local trueLabel: GenLabel = genGetLabel();
            printf("\tsubq \t%%%d, \t%%%d, \t%%%d\n", right, left, dest);
            printf("\t%s .L%d\n", condInstr(instr), trueLabel);
            printf("\tloadz \t0, \t%%%d\n", dest);
            genJump(doneLabel);
            genDefLabel(trueLabel);
            printf("\tloadz \t1, \t%%%d\n", dest);
            genDefLabel(doneLabel);
            return dest;
        }
        // ... as before
    }
    // ... as before
}
```

Testing the Comparison Operators. Create `xtest_gen_06.abc`:

```
@ "gen.hdr"
@ "symtab.hdr"

fn main()
{
    genInit();
    // global x: i64;
    genAddGlobal("x");
    // fn main() {
    genFuncDefBegin("main");
    local left: GenReg = genConstant("2");
    local right: GenReg = genConstant("1");
    genReleaseReg(left);
    genReleaseReg(right);
    local res: GenReg = genInstr(GEN_GT, left, right);
    genReturn(res);
    genReleaseReg(res);
    genFuncDefEnd();
}
```

Compile the test program, assemble the generated code, and execute it. The exit code should be `1`, since `2 > 1` is `true`. Now test all six comparison operators (`GEN_EQ`, `GEN_NE`, `GEN_LT`, `GEN_LE`, `GEN_GT`, `GEN_GE`) with different operands. Include cases where the left operand is smaller than, equal to, and greater than the right operand. Use the exit code to verify that each comparison produces `1` when true and `0` when false.

© Michael C. Lehn | CC BY-SA 4.0

Step 7 — Adding Comparison Operators to Expression Trees

Our code generator can now handle all six comparison operators. However, our expression trees do not yet support them. Let's change that!

First, extend the **binary operator group** of `ExprKind` in `expr.hdr`:

```
    EXPR_EQUAL,
    EXPR_NOT_EQUAL,
    EXPR_LESS,
    EXPR_LESS_EQUAL,
    EXPR_GREATER,
    EXPR_GREATER_EQUAL,
```

Next, modify the existing test program `xtest_expr_gen.abc`:

```
@ <stdio.hdr>
@ "expr.hdr"

fn main()
{
    local expr: -> Expr;
    expr = createBinaryExpr(EXPR_LESS,
                            createIntegerExpr(UStrCreate("20")),
                            createIntegerExpr(UStrCreate("22")));

    printf("\t# load expression: ");
    printExprTree(expr);
    printf("\n");
    genReleaseReg(loadExpr(expr));
    releaseExpr(expr);
}
```

Run the test. You should encounter an assertion because the new expression kind is not yet handled.

Tasks

1. **Extend** `printExprTree()` to handle the new comparison operators. Remember that we also use its output to generate expression trees with `make tree.tex`. To ensure that the operator symbols are rendered correctly by LaTeX, enclose them in curly braces:

   ```
   case EXPR_EQUAL: printf("{==}"); break;
   case EXPR_NOT_EQUAL: printf("{!=}"); break;
   case EXPR_LESS: printf("{<}"); break;
   case EXPR_LESS_EQUAL: printf("{<=}"); break;
   case EXPR_GREATER: printf("{>}"); break;
   case EXPR_GREATER_EQUAL: printf("{>=}"); break;
   ```

2. **Extend** `loadExpr()` to generate code for the new binary expression kinds. This should be straightforward: simply map each `ExprKind` to the corresponding `GenInstr`:

   ```
   case EXPR_EQUAL: return genInstr(GEN_EQ, left, right);
   case EXPR_NOT_EQUAL: return genInstr(GEN_NE, left, right);
   case EXPR_LESS: return genInstr(GEN_LT, left, right);
   case EXPR_LESS_EQUAL: return genInstr(GEN_LE, left, right);
   case EXPR_GREATER: return genInstr(GEN_GT, left, right);
   case EXPR_GREATER_EQUAL: return genInstr(GEN_GE, left, right);
   ```

3. **Test your implementation.** Run `xtest_expr_gen` again. It should execute without assertions, print the correct expression tree in the assembly comment, and generate the corresponding assembly instructions. Modify the test program to try all six new expression kinds.

Remember Worksheet 13?

We replaced separate parsing functions for different precedence levels with a single function, `parseBinary()`. This made our parser easy to extend with additional *left-associative* binary operators. Two helper functions do the work: `exprKindFromToken()` maps lexer tokens to expression kinds, while `tokenKindPrec()` assigns operator precedence.

In Worksheet 21, assignments required special treatment because `=` is *right-associative*. For our comparison operators, however, we can finally take full advantage of the design introduced in Worksheet 13: we only need to extend these two functions!

© Michael C. Lehn | CC BY-SA 4.0

Step 8 — Adding Comparison Operators to the Parser

Extend the two helper functions in `parseexpr.abc` as follows:

```
fn exprKindFromToken(token: Token): ExprKind
{
    switch (token.kind) {
        case PLUS: return EXPR_ADD;
        case MINUS: return EXPR_SUB;
        case ASTERISK: return EXPR_MUL;
        case SLASH: return EXPR_DIV;
        case PERCENT: return EXPR_MOD;
        case EQUAL2: return EXPR_EQUAL;
        case NOT_EQUAL: return EXPR_NOT_EQUAL;
        case LESS: return EXPR_LESS;
        case LESS_EQUAL: return EXPR_LESS_EQUAL;
        case GREATER: return EXPR_GREATER;
        case GREATER_EQUAL: return EXPR_GREATER_EQUAL;
        default:
            printf("token: %d (%s)\n", token.kind, tokenKindStr(token.kind));
            assert(0 && "internal error: exprKindFromToken");
            return 0;
    }
}

fn tokenKindPrec(kind: TokenKind): int
{
    switch (kind) {
        default: return 0;
        case ASTERISK: return 13;
        case SLASH: return 13;
        case PERCENT: return 13;
        case PLUS: return 12;
        case MINUS: return 12;
        case GREATER: return 10;
        case GREATER_EQUAL: return 10;
        case LESS: return 10;
        case LESS_EQUAL: return 10;
        case EQUAL2: return 9;
        case NOT_EQUAL: return 9;
    }
}
```

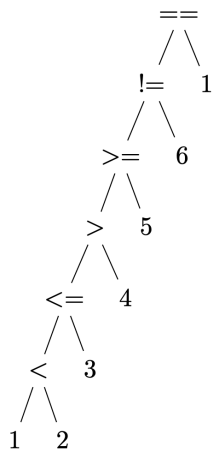
Notice that relational operators (`<`, `<=`, `>`, `>=`) have lower precedence than arithmetic operators but higher precedence than equality operators (`==`, `!=`).

Tasks

1. **Test the parser and visualize expression trees.** Use `make tree.tex` to inspect the tree generated for:

   ```
   1 < 2 <= 3 > 4 >= 5 != 6 == 1
   ```

   Check that the tree reflects the intended precedence and left associativity.



   Try additional expressions of your own, including combinations of arithmetic and comparison operators.

© Michael C. Lehn | CC BY-SA 4.0

2. **Update** `examples/ws22_readint.not-abc` to use comparison operators:

```
fn printInt(n)
{
    if (n == 0) {
        putchar('0');
    } else {
        if (n / 10 > 0) {
            printInt(n / 10);
        }
        putchar(n % 10 + '0');
    }
}

fn readInt()
{
    local n, ch;
    n = 0;
    ch = getchar();
    while (ch != '\n') {
        n = n * 10 + ch - '0';
        ch = getchar();
    }
    return n;
}

fn main()
{
    local n;
    n = readInt();
    printInt(n);
    putchar('\n');
}
```

What's Next?

Wouldn't it be nice to write something like:

```
if (ch >= '0' && ch <= '9') {
    // ...
}
```

Unfortunately, logical AND (`&&`) and OR (`||`) are not quite as straightforward as our other binary operators — at least not if we want short-circuit evaluation. We will see why in a later session.

© Michael C. Lehn | CC BY-SA 4.0